Bi-Weekly Research Progress Report

Project:	IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent:	Sarthak Mushrif
Reporting Period:	Weeks 1 & 2
Hours Logged:	20 Hours

1. Executive Summary of Progress

During the first two weeks of this research project, work focused on establishing the foundational development environment and initiating the design of the custom-built compiler frontend. Initial efforts were strictly allocated to the Lexical Analysis phase of the middleware, utilizing Compiler Design principles to tokenize raw SQL queries. Furthermore, initial AWS IAM (Identity and Access Management) policies and VPC structure planning were drafted in preparation for the infrastructure phase.

2. Detailed Task Breakdown (20 Hours)

- **Development Environment Initialization (4 Hours):** Configured the local Java (JDK 17) development workspace and established the build lifecycle using Maven to manage dependencies for the Netty and ANTLR4 libraries.
- **ANTLR4 Integration and Setup (4 Hours):** Integrated the ANTLR4 framework into the project repository to handle the generation of the Lexer and Parser for the SQL dialect.
- **Initial SQL Grammar Definition (8 Hours):** Drafted the initial `IsoShardSql.g4` grammar file to define the lexical tokens for basic SQL operations. The focus was specifically on `SELECT`, `FROM`, and `WHERE` clauses required for the planned Predicate Pushdown routing logic.
- **Abstract Syntax Tree (AST) Architecture (2 Hours):** Designed the base Java class structures for the AST that will eventually be traversed by the Visitor Pattern to identify query optimization opportunities.
- **AWS Infrastructure Pre-planning (2 Hours):** Drafted the architectural layout for the multi-tier distributed application to ensure secure communication channels between the future proxy layer and the AWS RDS physical partitions.

3. Challenges & Roadblocks

- **Grammar Ambiguities:** Encountered initial challenges with left-recursion in the ANTLR grammar when defining complex nested `WHERE` clauses. This was resolved by refactoring the expression rules to enforce strict operator precedence and associativity.

4. Objectives for Next Reporting Period (Weeks 3 & 4)

- Complete the parser rules for basic `INSERT` and `UPDATE` statements to support distributed write operations.
- Implement the base `SqlVisitor` Java class to traverse the generated AST.
- Develop unit tests to verify that the Lexer correctly tokenizes raw SQL strings without throwing syntax errors.

Appendix A: Technical Artifact - Initial ANTLR4 Grammar

The following `IsoShardSql.g4` grammar file was developed during this reporting period to generate the Lexer and Parser for the IsoShard Proxy SQL frontend.

```
grammar IsoShardSql;

// ============================================
// PARSER RULES (Abstract Syntax Tree Nodes)
// ============================================

statement
    : selectStatement EOF
    ;

selectStatement
    : SELECT columnList FROM tableName (WHERE condition)?
    ;

columnList
    : IDENTIFIER (',' IDENTIFIER)* | '*'
    ;

tableName
    : IDENTIFIER
    ;

condition
    : expression
    ;

// Handles left-recursion safely for WHERE clause evaluations
expression
    : left=expression operator=(EQ | NEQ | GT | LT | GTE | LTE) right=
    expression
    | IDENTIFIER
    | STRING_LITERAL
    | NUMERIC_LITERAL
    ;

// ============================================
// LEXER RULES (Tokenization)
// ============================================

// Keywords (Case-insensitive matching for standard SQL)
SELECT  : [sS][eE][lL][eE][cC][tT] ;
FROM    : [fF][rR][oO][mM] ;
WHERE   : [wW][hH][eE][rR][eE] ;

// Operators
EQ      : '=' ;
NEQ     : '!=' | '<>' ;
GT      : '>' ;
LT      : '<' ;
GTE     : '>=' ;
LTE     : '<=' ;

// Literals and Identifiers
IDENTIFIER      : [a-zA-Z_][a-zA-Z0-9_]* ;
STRING_LITERAL  : '\'' ~'\''* '\'' ;
NUMERIC_LITERAL : [0-9]+ ('.' [0-9]+)? ;

// Whitespace (Ignored by the parser)
```

```
58 WS : [ \t\r\n]+ -> skip ;
```

Listing 1: IsoShardSql.g4